Prova prática – Automação de Testes

Disciplina: Automação de Testes de Software

Linguagem: Python

Ferramentas: Pytest e unittest.mock

Quantidade: 6 exercícios

Valor: 10 pontos

Exercício 1 – Classificação de notas (1,5 ponto)

Crie uma função chamada `classificar_produto(quantidade)`.

O algoritmo deverá classificar a situação de um produto no estoque:

- Quantidade igual a zero: retornar `"Sem estoque"`.
- Quantidade entre 1 e 5: retornar `"Estoque baixo"`.
- Quantidade entre 6 e 20: retornar `"Estoque normal"`.
- Quantidade superior a 20: retornar `"Estoque alto"`.
- Quantidade negativa: retornar `"Quantidade inválida"`.

Testes unitários:

1. Produto sem estoque.
2. Produto com estoque baixo.
3. Produto com estoque normal.
4. Produto com estoque alto.
5. Quantidade negativa.
6. Valores nos limites das classificações.

Exercício 2 – Inversão de palavras (1,5 ponto)

Crie uma função chamada `inverter_palavras(texto)`.

O algoritmo deverá:

- Receber uma frase.
- Inverter a ordem das palavras.
- Retornar a frase com as palavras invertidas.
- Caso receba uma string vazia, retornar uma string vazia.

Exemplo:

Entrada: `"Python é muito interessante"`

Saída: `"interessante muito é Python"`

Testes unitários:

1. Frase com várias palavras.
2. Frase com apenas uma palavra.
3. Frase vazia.
4. Frase com espaços extras.
5. Frase com números e letras.

Exercício 3 – Validação de CPF simplificada (1,5 ponto)

Crie uma função chamada `validar_documento(documento)`.

O algoritmo deverá:

- Receber uma sequência de números representando um documento.
- Aceitar somente documentos com 11 dígitos.
- Permitir que o documento seja recebido com pontos e hífen (`123.456.789-00`).
- Remover os caracteres de formatação antes da validação.
- Retornar `True` quando possuir 11 dígitos numéricos.
- Retornar `False` caso contrário.

Atenção: Não é necessário implementar o cálculo dos dígitos verificadores reais do CPF.

Testes unitários:

1. Documento com 11 dígitos.
2. Documento formatado.
3. Documento com menos de 11 dígitos.
4. Documento com mais de 11 dígitos.
5. Documento contendo letras.
6. Documento vazio.

Exercício 4 – Geração de código de identificação com Mock (2 pontos)

Crie duas funções.

Função 1: `gerar_codigo()`

Deverá retornar um código fictício de identificação, por exemplo: `"USR-1234"`.

Função 2: `criar_identificacao(nome)`

Deverá:

- Receber o nome de uma pessoa.
- Chamar `gerar_codigo()`.
- Retornar uma mensagem no formato:

`Usuário Carlos cadastrado com código USR-1234`.

Testes unitários:

1. Verificar se a mensagem foi gerada corretamente.
2. Verificar se `gerar_codigo()` foi chamada.
3. Verificar se a função foi chamada apenas uma vez.
4. Utilizar Mock para simular diferentes códigos.
5. Verificar o comportamento com nomes diferentes.

Exercício 5 – Verificação de acesso com Mock (2 pontos)

Crie duas funções.

Função 1: `verificar_permissao(usuario)`

Deverá verificar se o usuário possui permissão e retornar `True` ou `False`.

Função 2: `acessar_sistema(usuario)`

Deverá:

- Receber o nome do usuário.
- Chamar `verificar_permissao()`.
- Retornar `"Acesso autorizado"` quando possuir permissão.
- Retornar `"Acesso negado"` quando não possuir permissão.

Testes unitários:

1. Usuário com permissão.
2. Usuário sem permissão.
3. Verificar se `verificar_permissao()` foi chamada.
4. Verificar se foi chamada com o usuário correto.
5. Utilizar Mock para simular os dois cenários.

Exercício 6 – Sistema de fila de atendimento com Mock (1,5 ponto)

Crie três funções.

Função 1: `obter_proximo_cliente()`

Deverá retornar o nome do próximo cliente da fila.

Função 2: `registrar_atendimento(cliente)`

Deverá simular o registro do atendimento e retornar `True` quando realizado.

Função 3: `atender_proximo_cliente()`

Deverá:

- Chamar `obter_proximo_cliente()`.
- Verificar se existe um cliente na fila.
- Se existir, chamar `registrar_atendimento(cliente)`.
- Se não existir cliente, retornar `"Fila vazia"`.
- Se o atendimento for realizado, retornar `"Atendimento realizado"`.
- Se o registro falhar, retornar `"Falha no atendimento"`.

Testes unitários:

1. Atendimento realizado com sucesso.
2. Fila vazia.
3. Falha ao registrar atendimento.
4. Verificar se `obter_proximo_cliente()` foi chamada.
5. Verificar se `registrar_atendimento()` foi chamada com o cliente correto.
6. Verificar se `registrar_atendimento()` não foi chamada quando a fila estiver vazia.